

QSOE

Quick and Secure Operating Environment

Design & Architecture

One userspace, two interchangeable microkernels

Document: QSOE-DSN

Revision: Rev A — Draft

Applies to: QSOE 0.1

Yuri Zaporozhets

June 2026

© 2026 Yuri Zaporozhets.

Licensed under CC BY-ND 4.0 — verbatim redistribution with attribution; no derivatives.

<https://creativecommons.org/licenses/by-nd/4.0/>

Contents

1	Introduction	1
1.1	What is QSOE?	1
1.2	The core bet: one userspace, two kernels	1
1.3	Why two kernels?	2
1.4	What QSOE deliberately does not implement	2
1.5	Heritage and licensing	2
1.6	Scope of this manual	2
2	System Architecture	4
2.1	The software stack	4
2.2	The OS seam: exactly three things	5
2.3	The shared userspace	5
2.4	Source-tree layout	5
2.5	Selecting a variant	6
2.6	What is shared, and what diverges	6
3	The Two Kernels	7
3.1	Skimmer (QSOE/N)	7
3.2	seL4 (QSOE/L)	8
3.3	Mechanism, not policy	8
3.4	Mapping QNX concepts to each kernel	8
3.5	Why the difference does not reach userland	9
4	taskman — The Central System Server	10
4.1	Role	10
4.2	The shared body and the per-kernel core	10
4.3	Service domains	11
4.4	Process management	11
4.5	Memory management	11
4.6	Path and namespace management	12
4.7	System configuration: the /sys model	12
4.8	Credentials	12
5	IPC: Channels, Connections, Messages, and Pulses	13
5.1	Channels and connections	13
5.2	The Send/Receive/Reply rendezvous	13
5.3	coids and rvids	14
5.4	Message and reply payloads	14
5.5	Pulses	14
5.6	Bulk transfers	14
5.7	Distributed transparency (forward-looking)	15

6	The Resource-Manager Model	16
6.1	What a resource server is	16
6.2	A clean-room framework	16
6.3	Providers, handles, and methods	17
6.4	The reply contract: synchronous or deferred	17
6.5	Servers stay above the kernel	18
6.6	Examples and the FreePascal projection	18
7	The C Library and Dynamic Linking	19
7.1	One library, a thin seam	19
7.2	Why dynamic linking is mandatory	19
7.3	The run-time loader	19
7.4	The split of state: library versus taskman	20
7.5	The QNX API is part of libc	20
8	Boot	21
8.1	The common prologue	21
8.2	QSOE/N: kernel plus initrd	21
8.3	QSOE/L: one self-contained image	21
8.4	Two strategies, one destination	22
8.5	The bootloader and the self-booting image	22

Chapter 1

Introduction

1.1 What is QSOE?

QSOE — the Quick and Secure Operating Environment — is a QNX-style operating environment for 64-bit RISC-V, built entirely under the Apache-2.0 license. It follows the QNX Neutrino tradition: a small kernel with everything else in userspace, synchronous message-passing IPC, and the resource-manager model for system services.

What makes QSOE unusual is that it is not one system but *two*, sharing a single userspace. The same QNX-style userland runs, byte-for-byte, on either of two interchangeable microkernels:

- **QSOE/L** — on **seL4**, the formally verified capability microkernel; and
- **QSOE/N** — on **Skimmer**, a from-scratch microkernel written for this project, derived in idiom from DragonFly BSD’s light-weight kernel threads (LWKT).

The target platform is RISC-V rv64 (Sv39). Development is on the QEMU `virt` machine; QSOE also boots on real hardware, the SiFive HiFive Unmatched (FU740).

1.2 The core bet: one userspace, two kernels

The central design bet of QSOE is that a complete QNX-style userspace can be made *100 % identical* across two very different kernels, with only a thin, well-defined seam differing between them.

Concretely, the per-kernel surface is exactly three things:

1. **The kernel** — Skimmer (QSOE/N) or seL4 (QSOE/L).
2. **taskman** — the central system server, the analogue of QNX’s `procnto`. It sits directly on its kernel (seL4 endpoints and capabilities, or Skimmer channels and messages) and is therefore genuinely different per system, even though both link a large shared body of OS-independent logic.
3. **The OS-dependent part of the C library** — the per-kernel syscall/IPC seam beneath an otherwise shared `libc`.

Everything else — the shell, drivers, utilities, the dynamic loader, and the entire OS-independent body of the C library — lives once, in the shared `quser` tree, and is compiled identically for both systems. A user program built for QSOE is a single dynamically-linked binary that runs on either variant; only the system’s `libc.so` differs at load time (Chapter 7).

This discipline is what the rest of this manual is really about: keeping the two systems converged at the userspace level while letting each kernel be exactly what it wants to be underneath.

1.3 Why two kernels?

The two kernels answer different questions, and QSOE deliberately refuses to choose between them.

seL4 brings a machine-checked proof of functional correctness and a capability model with the strongest isolation guarantees available in any production microkernel. Where the priority is assurance — a minimal, verified trusted computing base — QSOE/L is the answer.

Skimmer brings full control of the kernel itself: a from-scratch, preemptive, priority-driven, per-CPU design aimed at hard real-time, where the scheduler, the interrupt path, and the IPC fast path are ours to shape. Where the priority is determinism and direct hardware mastery, QSOE/N is the answer.

Because the userspace is shared, a program, a driver, or a resource manager written once is exercised on both — which keeps the QNX-style API honest and surfaces, at every milestone, any place where the two seams have quietly drifted apart.

1.4 What QSOE deliberately does not implement

A few classic Unix facilities are intentionally absent. These are design decisions, not gaps to be filled later:

- **No `fork()`.** Process creation is via `posix_spawn()` only. Without `fork()` there is also no payoff for copy-on-write, which is likewise a hard non-goal: each spawn maps fresh frames, and shared memory is explicit.
- **No `select()`.** `poll()` is the single multiplexer.
- **No `brk()`.** Low-level memory comes only through mmap-based calls.

These choices follow QNX practice and keep both kernels' memory and process models simple.

1.5 Heritage and licensing

QSOE is the unencumbered continuation of QRV, a finished port of QNX to RISC-V. QRV proved the hardware bring-up and the API shape; QSOE shares none of its proprietary code and is free to evolve. The whole of QSOE is Apache-2.0. Skimmer was modeled in structure and idiom on DragonFly BSD's LWKT and message-port subsystems, then written fresh; the dynamic loader is derived from FreeBSD's (BSD-2-Clause). Imported files preserve their original license headers.

The name was coined as a play on *QSO* — the ham-radio term for a two-way contact — between seL4 and QNX, plus *E* for “established” (as a QSO is) or, just as fittingly, for “Environment”.

1.6 Scope of this manual

This manual describes the *architecture* of QSOE: the structure shared by both variants and the precise points at which they differ. It is not a tutorial (see the User Guide, QSOE-USR) nor a programming reference (the Programming Book, QSOE-PRG, and the App Porting Guide, QSOE-APG).

The remaining chapters proceed from the outside in:

- Chapter 2 — the system architecture: the software layers, the OS seam, and exactly what differs between QSOE/N and QSOE/L.

- Chapter 3 — the two kernels, side by side, and how each realizes the QNX IPC model.
- Chapter 4 — taskman, the central system server.
- Chapter 5 — channels, connections, messages, and pulses.
- Chapter 6 — the resource-manager model.
- Chapter 7 — the C library and dynamic linking.
- Chapter 8 — the boot path of each variant.

Chapter 2

System Architecture

This chapter looks at QSOE as a whole: the layers of software between the hardware and an application, which of those layers are shared between the two variants and which are not, and how the source tree is organized to keep that split honest.

2.1 The software stack

Figure 2.1 shows the stack. Read from the bottom up, it is the familiar microkernel shape: a small kernel in supervisor mode, and everything else — the system server, the C library, drivers, and applications — in user mode, talking by messages.

What is particular to QSOE is the dashed line. Everything *above* it is shared, compiled once and identical on both variants. Everything *below* it is the per-kernel seam. The whole engineering discipline of the project is to keep that line as low and as thin as possible.

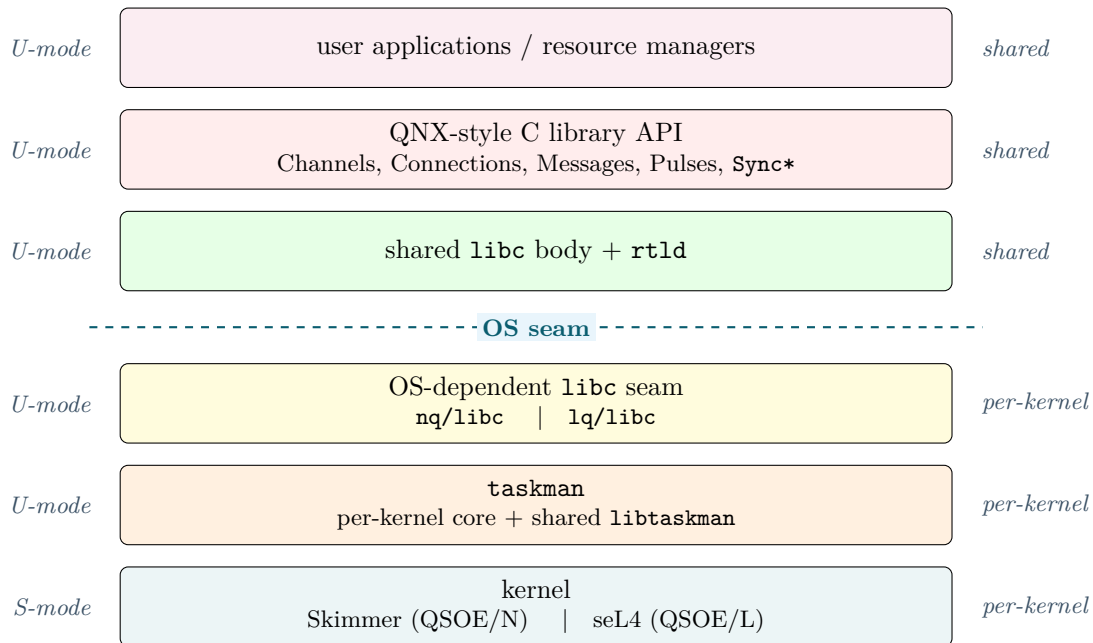


Figure 2.1: The QSOE software stack. Above the OS seam is shared and identical on both variants; below it is the thin per-kernel surface.

2.2 The OS seam: exactly three things

The per-kernel surface — everything below the dashed line — is exactly three things. Naming them precisely matters, because the promise of QSOE is that *nothing else* differs.

1. The kernel. Skimmer for QSOE/N, seL4 for QSOE/L. The two are not variations on a theme: Skimmer is a preemptive, priority-driven, per-CPU microkernel written for this project; seL4 is the formally verified capability kernel. They are described side by side in Chapter 3.

2. taskman. The central system server — QNX’s `procnto` analogue — sits directly on its kernel’s primitives and is therefore genuinely different per variant. But it is not written twice from scratch: the OS-independent body of every taskman (path management, credentials, system configuration, address-keyed synchronization, relocation) lives once in `libtaskman` and is linked into both. Only the core that touches the kernel — endpoints and capabilities on seL4, channels and messages on Skimmer — is per-variant. taskman is the subject of Chapter 4.

3. The OS-dependent part of libc. Beneath an otherwise shared C library, a thin seam implements the per-kernel syscall and IPC path: how a `MsgSend` actually reaches the kernel, how a pulse is delivered, how a thread traps. This seam lives in `nq/libc` and `lq/libc`, atop the shared `libc` body. The library as a whole is Chapter 7.

Everything not in that list — the dynamic loader, the QNX-style API surface, the entire OS-independent body of `libc`, and all of userland — is shared.

2.3 The shared userspace

The shared userspace lives in the `quser` tree: the shell, the utilities, the drivers, `sbin/init`, the resource-manager framework, and the OS-independent portion of the C library. It is compiled once and is *byte-for-byte* identical on both variants.

This is possible because user programs are **dynamically linked** by requirement, not by preference. Since the OS-dependent seam differs per kernel, a single portable binary cannot statically contain it; instead it binds the system’s `libc.so` at load time, through the run-time loader. A program built for QSOE is therefore one ELF that runs on either variant — the system supplies the right `libc.so`, and the dynamic loader (`rtld`) does the rest. This is why `rtld` is mandatory, not optional, in QSOE (Chapter 7).

2.4 Source-tree layout

The repository structure mirrors the architecture exactly: shared things live once, per-kernel things live under their variant.

Tree	Contents
<code>nq/</code>	QSOE/N: the Skimmer kernel and its taskman + <code>libc</code> seam
<code>lq/</code>	QSOE/L: the seL4 integration and its taskman + <code>libc</code> seam
<code>quser/</code>	the shared userspace — everything OS-independent
<code>libc/</code>	the shared C library tree (headers, body, <code>crt0</code> , <code>rtld</code>)
<code>libtaskman/</code>	the OS-independent body of every taskman

Table 2.1: Top-level source trees.

Two trees are worth a second look, because they are what make the shared/per-kernel split work in practice:

- `libc/` holds the C library *body* — the OS-independent code, the headers, `crt0`, and the run-time loader. Each variant’s `nq/libc` and `lq/libc` sources the shared body underneath their own seam, so the bulk of `libc` is written once.
- `libtaskman/` holds the OS-independent body of taskman, linked into both `nq/taskman` and `lq/taskman` (§2.2).

2.5 Selecting a variant

Each component owns its own Makefile; the top-level Makefile only descends into them. Building QSOE/N is a build under `nq/`; building QSOE/L is a build under `lq/`. Both produce the same shared `modpkg.cpio` userspace package via the shared `quser` recipe, and each adds only its own kernel-side artifacts on top.

Within a variant, board selection is a build-time choice (for QSOE/L, a Kconfig selection of one or more target boards: the QEMU `virt` machine, the SiFive Unmatched, or both). The boot path for each variant is the subject of Chapter 8.

2.6 What is shared, and what diverges

Table 2.2 is the whole architecture in one view: for each part of the system, whether it is shared or per-kernel. It is, in effect, the project’s contract with itself.

Component	Shared or per-kernel?
User applications, shell, utilities, drivers	shared
Resource-manager framework	shared
QNX-style C library API	shared
<code>libc</code> body, <code>crt0</code> , <code>rtld</code>	shared
<code>libtaskman</code> (taskman body)	shared
OS-dependent <code>libc</code> seam	per-kernel
<code>taskman</code> core	per-kernel
Kernel (Skimmer / seL4)	per-kernel

Table 2.2: The shared/per-kernel split, component by component.

The chapters that follow descend through the per-kernel rows of this table — the two kernels (Chapter 3) and taskman (Chapter 4) — and then back up through the shared rows: IPC (Chapter 5), the resource-manager model (Chapter 6), and the C library (Chapter 7).

Chapter 3

The Two Kernels

This is the one chapter that looks below the OS seam at the kernels themselves. Both present the same QNX-style message-passing model to the layers above; they realize it in very different ways. Skimmer is ours, written from scratch with a scheduler we control; seL4 is a formally verified capability kernel we build upon. The point of this chapter is to show that the *difference between them does not reach userland* — and exactly why.

3.1 Skimmer (QSOE/N)

Skimmer is a QNX-inspired microkernel for 64-bit RISC-V, skimmed in idiom from DragonFly BSD’s light-weight kernel threads (LWKT) and message-port subsystems, then grown — in layers — into a QNX-style API surface (`MsgSend/MsgReceive/MsgReply`, channels and connections, interrupt attachment). The DragonFly files were studied as a structural reference and then discarded; none of the current source is a derived work. The file layout and idiom echo DragonFly, but the code is ours and freshly written under Apache-2.0.

Being our own kernel, Skimmer is shaped by four design commitments:

1. **Per-CPU first; cross-CPU by message.** Per-hart state needs no locks; spinlocks exist only for genuinely cross-CPU paths. Pushing more state into a per-hart structure is cheaper than locking it.
2. **Preemptive, priority-driven scheduling.** The LWKT *mechanisms* are kept (per-CPU run queue, IPI-based migration); the LWKT *policy* (cooperative within a priority band) is replaced with strict priority preemption, so an interrupt service thread can displace a lower-priority thread on any hart immediately. This is what makes hard real-time possible (up to N hard-real-time tasks on an N -CPU system).
3. **Msgports are the transport, not the API.** The LWKT message ports are kernel-internal plumbing; the user-facing QNX `MsgSend` — rendezvous-only, with defined copy-/IOV semantics and priority inheritance — is built *on top of* them, not aliased to them.
4. **Personas are conceptual, not structural.** A user thread that traps into the kernel is the same thread: same TID, same priority, same run queue. Only the privilege level, address space, and stack pointer change. “User persona” and “kernel persona” never become two scheduler entities.

Userland sees Skimmer through a small, deliberately grouped syscall ABI (channels, messages, interrupts, timers, clocks, synchronization, threads). Because there is no QNX RISC-V ABI to preserve, the message layout, syscall numbers, and pulse format are free design choices. In keeping with a microkernel, the syscall set does not grow casually: new system functionality is added by sending messages to taskman, not by adding syscalls.

Skimmer has no demand paging and no copy-on-write (§1.4), and supports SMP from the start.

3.2 seL4 (QSOE/L)

seL4 is the formally verified capability microkernel. QSOE/L does not modify it; it builds the QNX programming model on top of seL4’s endpoints, notifications, and capability-managed address spaces. Only the seL4 kernel runs in supervisor mode; everything else — taskman included — is a user-mode process, giving QSOE/L a minimal, proven trusted computing base.

Where Skimmer hands us the scheduler, seL4 hands us a capability system: each process has its own CSpace and VSpace, every spawn building fresh page tables, and authority is conveyed only by capabilities. QSOE/L runs on the MCS configuration of seL4, which provides the scheduling contexts and native timeouts the QNX timer and synchronization surface needs.

3.3 Mechanism, not policy

The reason one userspace can sit on two such different kernels is a strict separation of *mechanism* from *policy*.

Each kernel provides the low-level *mechanism*: a way to rendezvous two threads and copy a message, a way to signal asynchronously, a handle that names a destination. Neither kernel decides the *policy*: who owns a channel, who allocates a connection, who revokes it on exit, how a name resolves to a server. That policy lives entirely above the seam — in taskman (Chapter 4) and the per-process C library (Chapter 7).

This is why the kernel mapping below describes only mechanism. Two kernels can realize the same QNX concept with different primitives precisely because the policy that gives those primitives their QNX meaning is written once, above them both.

3.4 Mapping QNX concepts to each kernel

Table 3.1 gives the correspondence: for each QNX concept, the mechanism that realizes it on each kernel. It is the concrete form of §3.3.

QNX concept	Skimmer (QSOE/N)	seL4 (QSOE/L)
Channel	Kernel channel object	Endpoint
Connection (ConnectAttach)	Connection to a channel, named by a coid	Capability to an Endpoint
MsgSend / MsgReceive / MsgReply	Rendezvous message syscalls, carried over LWKT msgports	Synchronous IPC (seL4_Call / seL4_Recv / seL4_Reply)
Pulse (MsgSendPulse)	Asynchronous pulse message	Notification
Connection ID (coid)	Process-local connection handle	Capability badge
Receive ID (rcvid)	Receive handle for the pending reply	Badge value on the incoming message
Channel ownership	Held by taskman	Master Endpoint capability held by taskman
Resource manager	User-space server	User-space server holding capabilities

Table 3.1: QNX concepts and the mechanism that realizes each, per kernel.

The table describes the *mechanism* only. It does not say who holds which handle, who allocates, or who revokes — that separation is deliberate, and the policy is the subject of the chapters above the seam.

One difference is worth naming, because it is visible to an architect even if not to an application: how a channel is represented. On Skimmer a channel is a global kernel object identified by a channel id; on seL4 it is a per-process Endpoint reached through a capability. The QNX programming model is faithfully preserved on both — code to the per-process model, discover servers by name, and request a global channel only when you explicitly mean to — so the distinction stays an implementation detail beneath the C library.

3.5 Why the difference does not reach userland

Above the seam, none of this is visible. taskman wraps the kernel mechanism into the QNX policy; the C library seam wraps the syscall path into the QNX API. An application calls `MsgSend` and receives a reply with identical semantics on either kernel — the rendezvous, the copy semantics, the priority inheritance are all specified by QSOE, not by the kernel underneath.

The remaining chapters live above the seam and are therefore written once for both variants: taskman (Chapter 4), the IPC model in full (Chapter 5), the resource-manager framework (Chapter 6), and the C library that presents it all (Chapter 7).

Chapter 4

taskman — The Central System Server

If the kernel is the mechanism, **taskman** is where the policy begins. It is the first user process the system starts, and every other process descends from it. This chapter describes what **taskman** is responsible for, how it is split between a shared body and a per-kernel core, and the main service domains it provides.

4.1 Role

taskman is QSOE's root server, and the direct analogue of QNX's **procnto**: a single userspace process that combines the responsibilities of a process manager, a memory manager, a path (namespace) manager, and a provider of system services.

The crucial difference from QNX is structural. In QNX, **procnto** is fused with the microkernel into one privileged image. In QSOE, the kernel provides only the bare mechanism (Chapter 3), and **taskman** is an ordinary user-mode process that implements all higher-level OS policy from outside the kernel. It is powerful by virtue of the capabilities and registries it holds, not by virtue of privilege: on QSOE/L it runs in U-mode like any other process; on QSOE/N likewise.

4.2 The shared body and the per-kernel core

taskman is the most variant-specific component in userland — it sits directly on its kernel's primitives — yet it is not written twice. Its OS-independent logic lives once in **libtaskman** and is linked into both **nq/taskman** and **lq/taskman**:

- **pathmgr** — the pathname space: registration, resolution, mounts, and symlink expansion (§4.6).
- **cred** — the credential model and the privilege policy (§4.8).
- **syscfg** — system configuration and the **/sys** model (§4.7).
- **sync** — the address-keyed wait/wake that backs the QNX **Sync*** API.
- **reloc** — relocation support for loaded images.

Only the *core* that touches the kernel is per-variant: endpoints, capabilities, and untyped memory on seL4; channels, messages, and frames on Skimmer.

A bootstrap paradox (QSOE/L only). This is a place where the mechanism-vs-policy split of Chapter 3 becomes concrete. On QSOE/L a channel is an seL4 Endpoint that taskman manages by *policy*, so `ChannelCreate` is implemented as a message *to* taskman — which creates a chicken-and-egg problem at startup: taskman cannot use `ChannelCreate` to make its own first channel, because no channel exists yet to carry the request. It therefore creates its primary Endpoint by hand, directly on seL4, and registers itself as the first entry (`pid=1`) in the channel registry; from then on every channel is created through the normal API path. On QSOE/N the question never arises: a channel is a Skimmer kernel *mechanism*, so `ChannelCreate` is an ordinary system call and taskman simply calls it like any other process.

4.3 Service domains

taskman multiplexes several service domains over its primary channel, dispatching on the message type (Table 4.1). Each corresponds to a responsibility QNX folds into `procnto`.

Domain	Operations	QNX equivalent
Process	spawn, terminate, wait/reap	<code>procnto</code> process manager
Memory	<code>mmap</code> , <code>munmap</code> , shared memory	<code>procnto</code> memory manager
Pathname	registration, resolution, <code>open</code> routing	<code>procnto</code> path manager
System	system info, configuration, shutdown	various <code>procnto</code> calls

Table 4.1: taskman service domains.

All interaction with taskman follows the QNX Send/Receive/Reply pattern: a client packs a request, sends it to taskman’s channel, and blocks for the reply. taskman runs a receive–dispatch–reply loop, handling one request and immediately waiting for the next. The IPC model itself is the subject of Chapter 5.

4.4 Process management

QSOE has no `fork()` (§1.4); process creation is via `posix_spawn()` only. A spawn is a single transaction with taskman: the client supplies the executable path and the new process’s arguments and environment, the latter through a small side-channel page rather than inline in the message. taskman loads the image, builds a fresh address space, sets up the new process’s first thread, and returns its `pid`.

Because there is no `fork()`, there is also no copy-on-write: each spawn maps *fresh* frames (§1.4). How those frames are obtained is the per-kernel part: on QSOE/L taskman retypes untyped memory into a new CSpace and VSpace under a per-process budget; on QSOE/N it allocates and maps fresh frames directly. The result is the same — a new, isolated process — reached two different ways below the seam.

When a process exits, taskman reaps it: it reclaims the process’s frames, capabilities, and registry entries, retires its channels and connections, and makes its exit status available to `waitpid()`. A faulting process is torn down the same way, with its peers notified so a blocked `MsgSend` to it fails cleanly rather than hanging.

4.5 Memory management

Low-level memory is obtained only through `mmap`-based calls; there is no `brk()` (§1.4). `mmap` and `munmap` route through taskman, which owns the per-process address space. The allocator is a bump cursor over the virtual address range: `mmap` advances the cursor and backs the new range with fresh frames; `munmap` releases the frames but does not rewind the cursor. Shared

memory is always explicit — there is no implicit sharing to inherit, precisely because there is no `fork()`.

4.6 Path and namespace management

taskman (through `libtaskman`'s `pathmgr`) owns the system's single pathname space. Servers register the path prefixes they are responsible for; when a client opens a path, taskman resolves it — following mounts and symlinks — to the server that should handle it, and the client is connected to that server. This is the mechanism by which a program reaches a resource manager *by name* without knowing where it lives; the resource-manager framework that sits on the other end is Chapter 6.

4.7 System configuration: the `/sys` model

Rather than answer a dedicated configuration IPC for every query, taskman exposes system facts through a small read-only model surfaced under `/sys` (for example `/sys/osname`, `/sys/version`, `/sys/board`). The model itself lives once in `libtaskman`; each variant feeds it from its own source of truth — the kernel-built information page on QSOE/N, the equivalent configuration path on QSOE/L — so the same `/sys` entries appear on both systems. A `uname`-style tool, or `getty`'s banner, reads these the same way regardless of kernel.

4.8 Credentials

taskman holds the credential model (user and group identity) and enforces the privilege policy in one place, in `libtaskman`'s `cred` module. The policy is the conventional one: the superuser (effective uid 0) is unrestricted, while an unprivileged process may only assume identities it already holds — its own real, effective, and saved ids — so it can shuffle among them but can never gain a new one, and in particular cannot `setuid(0)`. Because the rule lives in the shared body, both variants' taskman enforce it identically.

Chapter 5

IPC: Channels, Connections, Messages, and Pulses

Message passing is the heart of a QNX-style system, and of QSOE. Almost everything a program does that is not pure computation — opening a file, reading a device, asking taskman to spawn a process — is a message to a server. This chapter describes the model: how endpoints are named, how a synchronous message exchange works, and how asynchronous pulses complement it. The model is identical on both variants; the kernel mechanisms that realize it were given in Chapter 3.

5.1 Channels and connections

Communication is between a *channel* (the server side) and a *connection* (the client side).

A server creates a channel with **ChannelCreate** and then waits on it to receive messages. A client reaches that server by establishing a connection to the channel with **ConnectAttach**, which yields a *connection ID* (coid) — a small integer the client uses thereafter to send. The asymmetry is deliberate: one channel, many connections; the server receives, the clients send.

A client rarely calls **ConnectAttach** with raw coordinates. Instead it opens a *name*: taskman's path manager (§4.6) resolves the name to the owning server and arranges the connection. This is how a program reaches `/dev/console` or a filesystem without knowing which process serves it.

5.2 The Send/Receive/Reply rendezvous

The core exchange is synchronous and unbuffered — a rendezvous between two threads:

1. The client calls **MsgSend** on a coid and **blocks**.
2. The server's **MsgReceive** returns the message together with a *receive ID* (rcvid) naming this particular pending exchange.
3. The server processes the request and calls **MsgReply** with a status and an optional reply payload.
4. **MsgReply** unblocks the client; **MsgSend** returns the status the server passed.

There is no message queue and no buffering in the middle: if no server thread is receiving, the sender simply waits, and vice versa. A QNX implementation classifies a thread's blocked state precisely by which half of the rendezvous it is waiting on — SEND-blocked (the server has

not yet received), REPLY-blocked (received, not yet replied), or RECEIVE-blocked (a server idle with no client) — and QSOE preserves those semantics.

Because the exchange is synchronous, it carries priority with it: while a server handles a client's message it runs at the client's priority, so a high-priority client is not held off behind lower-priority work in the server. This priority inheritance is part of the contract, specified by QSOE rather than inherited from the kernel underneath.

5.3 coids and rvoids

The two handles name the two ends of an in-flight exchange. A **coid** is the client's process-local handle for a connection; a **rvoid** is the server's handle for a received-but-not-yet-replied message, the token it must present to `MsgReply`.

Following QNX faithfully, connection IDs and file descriptors share *one* integer namespace: a coid *is* an fd and an fd *is* a coid. An `open` returns a descriptor that is also a connection one can `MsgSend` on directly, and there is no separate descriptor table to keep in step — which is what lets the same `read/write/close` calls work uniformly on files, devices, and raw connections. Also as in QNX, a reserved *side-channel* range is set aside for the library's own internal connections (for instance the standing connection to taskman) so they never collide with application descriptors.

5.4 Message and reply payloads

A message is a block of bytes whose meaning is a contract between client and server; the first word is conventionally a message type that the server dispatches on. Both the send and the reply may be gathered from, or scattered into, several buffers (an I/O vector), so a header and a body in separate buffers need no prior concatenation.

The reply is *pure payload*. The server's completion status is not smuggled into the reply bytes: `MsgReply` takes the status as a separate argument, and the client receives it as the return value of `MsgSend`. Keeping status and payload distinct means a server can report an error with no reply body, and a client always knows the outcome without parsing the data.

5.5 Pulses

Not every notification wants a reply. A *pulse* is a small, fixed-size, asynchronous message: the sender does not block and there is no reply. Pulses carry a code and a value, and are the right tool for events and readiness notifications — a driver telling a server that data has arrived, a timer firing.

Pulses also carry **signals**. QSOE delivers POSIX signals as pulses to a per-process *system thread*: every program runs, from the first instant, two threads — its `main` thread and a system thread that receives these pulses and runs signal handlers. A consequence worth stating plainly is that Skimmer has *no* `Signal*` system call: signal delivery is pulse delivery, with no separate kernel mechanism. The same model holds on QSOE/L.

5.6 Bulk transfers

The rendezvous is tuned for the common case of small messages, but some exchanges are large — reading a megabyte from a file, say. For these, QSOE carries the payload in bulk, copying it page by page directly between the two address spaces rather than forcing the application to set up shared memory by hand. A generous upper bound keeps a single transfer from being unbounded. The application sees an ordinary large `MsgSend`; the page-by-page movement is below the API.

5.7 Distributed transparency (forward-looking)

Several IPC entry points carry a *node descriptor* argument that, today, always names the local node. These are not vestigial: they are deliberate placeholders for transparent distributed message passing — the ability for a connection to reach a server on another node with the same Send/Receive/Reply semantics — which is planned work beyond 1.0. The arguments thread through the IPC surface now so that distribution can arrive later without breaking the ABI. They should never be removed as “unused.”

Chapter 6

The Resource-Manager Model

In a microkernel, a file is not a kernel object — it is a conversation with a server. So are a device, a directory, and a pseudo-terminal. The *resource-manager model* is the pattern those servers follow, and it is how QSOE presents a POSIX face (`open`, `read`, `write`, `stat`, ...) over message passing. This chapter describes the model and the framework QSOE provides for writing it.

6.1 What a resource server is

A *resource server* is an ordinary process that registers one or more paths in the system namespace and answers client I/O on them. A word on terms, used consistently throughout: following QNX, the *pattern* is the *resource-manager model*, while a concrete process that implements it is a *resource server* — the model is a manager, a thing built to it is a server. (taskman’s *path manager* of §4.6, and its process and memory managers, are unrelated components that merely share the word.)

A resource server ties together the two preceding chapters: it registers its paths with taskman’s path manager (§4.6), and it serves clients over the Send/Receive/Reply IPC of Chapter 5.

The client never sees any of this. When a program calls `open("/dev/console", ...)`, the path manager resolves the name to the console server and connects the client to it; the returned descriptor is a connection (§5.3), and the program’s subsequent `read` and `write` are messages to that server. “Everything is a file” becomes “everything is a server you reach by name.”

6.2 A clean-room framework

QNX programmers will recognize the model; QSOE does *not* reuse QNX’s implementation of it. The QNX `resmgr/iofunc` layer is proprietary and is never ported. QSOE instead provides its own framework, designed from scratch, that keeps the familiar shape while making two deliberate choices of its own:

1. **A server is expressed as objects.** Each resource is one C structure — a *Provider* — carrying both the resource’s data and a pointer to its method table. The model is defined so that one Provider projects cleanly onto a single object: its method table is that object’s virtual-method table. A server author writes objects, not message handlers.
2. **The framework owns the loop.** The receive–dispatch–reply loop belongs to the framework. It decodes each incoming request into *typed* arguments and invokes the one method that names the operation; a method never sees an undifferentiated message buffer. There is no reachable raw buffer to mis-parse or overrun.

6.3 Providers, handles, and methods

Three concepts carry the model.

Provider. The resource object. It holds the resource’s POSIX attributes (its “inode”: mode, owner, size, timestamps), its registered path, and — by subtyping, embedding the base as its first member — whatever private state the resource needs. Its first member is the pointer to its method table, shared by all instances of the same type.

Handle. One open instance, created by the framework when an open is accepted and destroyed after the matching close. It carries the per-open state: the current offset, the accepted flags, and an author pointer for anything else. One Handle per successful **open**.

Methods. The Provider’s method table groups the operations a resource can answer. Each takes the Provider as an explicit *Self* argument, so the same table serves every instance:

- **lifecycle** — **acquire/release**: admit or refuse an open, and drop its state on close;
- **leaf I/O** — **pull/push/seek**: read, write, reposition;
- **attributes** — **query/adjust**: **stat**, and **chmod/chown/truncate/utime**;
- **control and readiness** — **control** (the typed, bounded escape hatch in place of QNX’s **devctl**) and **ready** (which **poll** events are satisfiable now);
- **asynchronous** — **service/cancel**: an interrupt or internal event, and a client abandoning a blocked request;
- **directory** — **lookup/list/make/unlink**, present only on Providers that manage a namespace subtree.

A method slot may be left empty: a NULL slot has a defined meaning, and the framework supplies *correct-by-default* base methods (permission checks in **acquire**, attribute copies in **query**, offset arithmetic in **seek**). A minimal server therefore overrides only **pull** and **push** and inherits the rest.

6.4 The reply contract: synchronous or deferred

A request-bearing method produces *exactly one* reply, in one of two ways. It may return **synchronously** — the count read, the new offset, or a negative **errno** — and the framework turns that into the reply at once. Or it may **defer**: return a sentinel saying “not yet,” and complete the call later.

Deferral is what makes drivers possible. A serial read with no data waiting cannot answer immediately; the driver defers it, and when the receive interrupt arrives, its **service** method completes the parked call with the bytes. The in-flight call is named by an *opaque* token: the author can complete or cancel it but cannot reach the underlying message, so there is no raw buffer to corrupt — a deliberate improvement over the exposed context object of the QNX framework.

6.5 Servers stay above the kernel

One discipline keeps the model portable: a resource server *never* calls the kernel directly. It uses only POSIX and the QSOE IPC primitives, never seL4 or Skimmer system calls. This is what lets the very same server binary run on both variants — the resource servers live entirely above the OS seam (Chapter 2), in the shared userspace.

6.6 Examples and the FreePascal projection

The framework is the single way servers are built: the `/dev/console` server, the serial-port driver, and the read-only filesystems that mount the boot archive and the on-disk `fs-qrv` are all Providers with method tables, differing only in which methods they implement.

The object orientation is not incidental. The model is specified so that one Provider maps to exactly one FreePascal `object` and its method table to that object's virtual-method table — so a resource server can be written naturally in FreePascal as well as C. The full data types, the per-method contract, and the request-message (`_IO_*`) layer beneath the framework are the subject of the Programming Book (QSOE-PRG); this chapter has described only their shape.

Chapter 7

The C Library and Dynamic Linking

The C library is where the whole system meets the application. Every chapter so far has described something an application reaches *through* libc: the IPC primitives (Chapter 5), the path-based `open` that finds a resource server (Chapter 6), the requests to taskman (Chapter 4). It is also where the last per-kernel difference hides — in a thin seam beneath an otherwise shared library.

7.1 One library, a thin seam

The C library is mostly shared. Its OS-independent body — the standard C functions, `stdio`, the memory allocator, the string and math routines, and the QNX-style API surface that presents Channels, Connections, Messages, and Pulses — lives once in the `libc` tree and is compiled for both variants. Around 85% of the library is shared at the source level.

Beneath it sits the per-kernel *seam* (`nq/libc` and `lq/libc`): the small layer where the library actually meets the kernel. This is where a `MsgSend` turns into the right trap or capability invocation, where a thread is created, where a pulse is delivered, and where the few per-variant data shapes are fixed. The seam is deliberately small — it is the third and last of the three things that differ between the variants (§2.2).

The library tree also carries the C runtime startup (`crt0`) and the run-time loader, so that everything needed to start and link a program lives in one place.

7.2 Why dynamic linking is mandatory

Chapter 2 stated that user programs are dynamically linked by requirement; here is why, precisely. Because the seam differs per kernel, the machine code that reaches the kernel is *not the same* on QSOE/N and QSOE/L. A statically linked binary would have to bake one kernel’s seam into itself, and would then run on only that variant.

So a QSOE program contains no libc of its own. It is one position-independent ELF that, at load time, binds the *system’s* `libc.so` — the one built for whichever kernel it is running on. The same binary therefore runs on either variant: the system supplies the correct library. This is the technical content of the “one userspace, two kernels” promise at the level of an individual executable, and it is why a run-time loader is not optional in QSOE but a mandatory part of the system.

7.3 The run-time loader

The run-time loader (`rtld`) is derived from FreeBSD’s, with its BSD-2-Clause heritage preserved. At program start it maps the executable and its libraries, resolves their symbols, and applies

relocations, then transfers control to the program's entry point.

One choice matters for QSOE's real-time ambitions: binding is *eager*. Rather than resolving each function on its first call (the usual lazy-PLT scheme), the loader resolves every symbol up front. Deferring symbol resolution to first call would put unbounded work — and a possible fault — in the middle of a time-critical path; resolving eagerly keeps the running program's timing predictable, which a hard-real-time system cannot do without.

7.4 The split of state: library versus taskman

A QNX-style system divides its bookkeeping between the per-process C library and the central server, and QSOE follows that division. Some state is private to a process and lives in its library: the table of its open connections, its side of the coid namespace, its signal dispositions. Other state is system-wide and lives in taskman: the channel registry, the process table, the pathname space.

The guiding rule is to keep in the library whatever can correctly be kept there, so that a common operation does not require a message to taskman for every step. A **sigaction**, for instance, is a process-local update in the library — taskman never sees a signal disposition — while a **spawn**, which must allocate a new process system-wide, is necessarily a request to taskman. This is the same mechanism-versus-policy line drawn one level higher: the library holds the policy that is safely process-local; taskman holds the policy that must be central.

7.5 The QNX API is part of libc

In QNX the message-passing API is simply part of the C library, and QSOE arranges the same. The QNX-style surface — **ChannelCreate**, **ConnectAttach**, **MsgSend** and its family, the **Sync*** primitives, the timer and thread calls — is built into **libc.so** alongside the POSIX functions, not delivered as a separate archive a program must remember to link. A program links one library and has both the standard C interface and the QNX system interface.

Chapter 8

Boot

This final chapter follows each variant from power-on to `/sbin/init`. The two paths share a prologue and a destination but differ in the middle — and the way they differ says something about the two kernels’ natures. The chapter closes with the bootloader and the self-booting disk image that are common to both.

8.1 The common prologue

Both variants begin the same way. The platform firmware — OpenSBI in M-mode — initializes the hardware and hands control to S-mode with a pointer to the flattened device tree (FDT) in a register. The FDT is the system’s self-description: the harts and their capabilities, the memory map, the interrupt controller, the console and the timer. Both kernels are FDT-driven, reading it to discover the machine rather than being compiled for one board.

From here the two paths diverge in how the kernel and the first userland are packaged.

8.2 QSOE/N: kernel plus initrd

QSOE/N boots in the conventional kernel-plus-initrd shape. Two artifacts are involved:

- `skimmer.bin` — the Skimmer kernel as a *flat RISC-V Image*, not an ELF. The loader jumps straight to the image’s entry, as it would for a Linux **Image**.
- `modpkg.cpio` — the shared userspace package, supplied separately as an initial RAM disk.

OpenSBI enters `skimmer.bin`; the kernel finds the `initrd` by reading its location from the `/chosen` node of the FDT, parses the `cpio` archive, and starts `taskman` from it, which goes on to `/sbin/init`. Under QEMU the `initrd` is handed over with `-initrd`; under the `mr-bml` bootloader it is loaded by a `modpkg` directive. Nothing is embedded — kernel and `initrd` remain two files, joined only at boot through the device tree.

8.3 QSOE/L: one self-contained image

QSOE/L boots from a single self-contained ELF, assembled by nesting each stage inside the previous one:

1. OpenSBI enters the **elfloader**, a small loader that knows how to unpack and start an **seL4** system.
2. The `elfloader` image embeds an archive of the **seL4 kernel** and **taskman** (the root task), included directly into its data at build time.

3. `taskman`, in turn, embeds the userspace package, so that once it runs it can start `/sbin/init` with no external file.

The result is one ELF — the `qsoe-1-*.elf` of the downloads — that needs nothing alongside it to reach a shell. This nesting reflects `seL4`’s nature: its initial set of resources and the root task are fixed at build time and bootstrapped through capabilities, so packaging the whole system as one statically assembled image is the natural fit.

8.4 Two strategies, one destination

The contrast is deliberate (Table 8.1). QSOE/N keeps the kernel and the `initrd` as separate artifacts discovered through the device tree — the familiar, flexible shape. QSOE/L folds everything into one ELF, matching the static, capability-bootstrapped way an `seL4` system comes up. Both arrive at the same place: `taskman` running, and `/sbin/init` starting the shared userspace described in the rest of this manual.

	QSOE/N (Skimmer)	QSOE/L (seL4)
Kernel form	flat RISC-V Image (<code>skimmer.bin</code>)	ELF, via <code>elfloader</code>
Userspace	separate <code>initrd</code> (<code>modpkg.cpio</code>)	embedded in the image
Joined	at boot, via the FDT <code>/chosen</code>	at build, by nesting
Artifacts	two files	one self-contained ELF

Table 8.1: The two boot strategies.

8.5 The bootloader and the self-booting image

On real hardware, and in the published disk image, a bootloader sits before all of this. QSOE uses **mr-bml**, its own GRUB-derived UEFI bootloader: an EFI application that reads ext-formatted partitions, presents a menu, and loads either variant — `kernel` plus `modpkg` for QSOE/N, or the self-contained ELF for QSOE/L.

This is what makes the downloadable image self-booting. The disk carries `mr-bml` in its EFI System Partition and the kernels on an ordinary filesystem, so the firmware runs `mr-bml`, `mr-bml` shows the QSOE menu, and a chosen variant boots — with no external kernel argument, under QEMU exactly as on the Unmatched. The mechanics of building and running that image belong to the User Guide (QSOE-USR); here it is enough to see where the bootloader fits: in front of the two boot paths this chapter has described, feeding each the artifacts it expects.